

**Id no:25dce049**

## **Practical 7: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage**

**Problem Definition** Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages.

### **Key Questions / Analysis / Interpretation**

- 1. Is the form submitted using POST?**
- 2. Are inputs validated and sanitized server side?**
- 3. Is file writing handled safely?**
- 4. Are success and error responses displayed clearly?**

### **Key Skills to be Addressed**

**PHP forms, POST handling, server-side validation, sanitization, file writing, CSV/JSON storage**

**Applications** Backend form processors and small data collection systems  
**Learning Outcome** Students will implement server-side form processing with safe file-based

**storage.**

**CO Mapping** CO1, CO5

**Dataset/Test Data** Registration/contact form inputs

**Tools/Technology to be  
Used**

**PHP, XAMPP/WAMP/LAMP, browser dev tools**

**Total Hours Implementation:** 4 hours | **Total engagement:** 6 hours

**Post Laboratory Work**

**Description**

**Submit PHP source code, test cases, and generated CSV/JSON file.**

## Evaluation Strategy Including Viva

POST handling, validation, sanitization, storage correctness, and viva.

### Advanced/Intermediate Extension

Intermediate: Display stored CSV/JSON records on a webpage.

Advanced: Add CSRF token validation for form submission.

### Code:

#### 1.index.php:

```
<?php
/**
 * Practical 7 - Registration form.
 * Displays success messages flashed from process.php and the input form
 * itself.
 */

declare(strict_types=1);

require_once __DIR__ . '/includes/bootstrap.php';

$success = $_SESSION['flash_success'] ?? '';
unset($_SESSION['flash_success']);

$errors = $_SESSION['flash_errors'] ?? [];
$old = $_SESSION['flash_old'] ?? [];
$hasData = isset($_SESSION['flash_success']) || $errors !== [];
unset($_SESSION['flash_errors'], $_SESSION['flash_old']);
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title><?= e(APP_NAME) ?></title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<main class="card">
    <h1>Student Registration</h1>
    <p class="lead">Practical 7 - PHP form processing with server-side
validation and CSV/JSON file storage.</p>

    <?php if ($success !== ''): ?>
        <div class="alert alert-success"
role="status"><strong>Success!</strong> <?= e($success) ?></div>
    <?php endif; ?>

    <?php if ($errors !== []): ?>
        <div class="alert alert-error" role="alert">
            <strong>Error!</strong> Please fix the <?= count($errors) ===
1 ? 'field' : 'fields' ?> below.
        </div>
    <?php endif; ?>

    <form id="form" method="post" action="process.php" novalidate>
        <?= Csrft::field() ?>

        <div class="field<?= isset($errors['full_name']) ? ' has-error' :
'' ?>">
            <label for="full_name">Full name <span
class="req">*</span></label>
            <input type="text" id="full_name" name="full_name"
maxlength="60" required
                value="<?= e($old['full_name']) ?? '' ?>">
            <?php if (isset($errors['full_name'])): ?><p class="error"><?=
e($errors['full_name']) ?></p><?php endif; ?>
        </div>

        <div class="field<?= isset($errors['email']) ? ' has-error' : ''
?>">
            <label for="email">Email <span class="req">*</span></label>
            <input type="email" id="email" name="email" maxlength="120"
required

```

```

        value="<?= e($old['email'] ?? '') ?>">
        <?php if (isset($errors['email'])): ?><p class="error"><?=
e($errors['email']) ?></p><?php endif; ?>
    </div>

    <div class="field<?= isset($errors['phone']) ? ' has-error' : ' '
?>">
        <label for="phone">Phone number <span
class="req">*</span></label>
        <input type="text" id="phone" name="phone" maxlength="15"
required
        value="<?= e($old['phone'] ?? '') ?>">
        <?php if (isset($errors['phone'])): ?><p class="error"><?=
e($errors['phone']) ?></p><?php endif; ?>
    </div>

    <div class="row">
        <div class="field<?= isset($errors['course']) ? ' has-error' :
' ' ?>">
            <label for="course">Course <span
class="req">*</span></label>
            <select id="course" name="course" required>
                <option value="">-- Select course --</option>
                <?php foreach (ALLOWED_COURSES as $course): ?>
                    <option value="<?= e($course) ?>"<?=
($old['course'] ?? '') === $course ? ' selected' : ' ' ?>><?= e($course)
?></option>
                <?php endforeach; ?>
            </select>
            <?php if (isset($errors['course'])): ?><p
class="error"><?= e($errors['course']) ?></p><?php endif; ?>
        </div>

        <div class="field<?= isset($errors['year']) ? ' has-error' :
' ' ?>">
            <label for="year">Year of study <span
class="req">*</span></label>
            <select id="year" name="year" required>
                <option value="">-- Select year --</option>
                <?php foreach (ALLOWED_YEARS as $year): ?>

```

```

        <option value="<?= e($year) ?>"<?= ($old['year']
?? ' ') === $year ? ' selected' : ' ?><?= e($year) ?></option>

        <?php endforeach; ?>
    </select>
    <?php if (isset($errors['year'])): ?><p class="error"><?=
e($errors['year']) ?></p><?php endif; ?>
</div>
</div>

<div class="field"<?= isset($errors['message']) ? ' has-error' : '
?>">
    <label for="message">Message (optional)</label>
    <textarea id="message" name="message" rows="4" maxlength="500"
        placeholder="Anything you would like to add"><?=
e($old['message'] ?? ' ') ?></textarea>
    <?php if (isset($errors['message'])): ?><p class="error"><%=
e($errors['message']) ?></p><?php endif; ?>
</div>

<div class="actions">
    <button type="submit" class="btn btn-primary">Submit
registration</button>
    <a class="btn" href="records.php">View stored records (<%=
e((string) Storage::count()) ?>)</a>
</div>
</form>
</main>
</body>
</html>

```

## 2.records.php:

```

<?php
/**
 * Practical 7 (Intermediate extension) - Display the stored CSV/JSON
records.
 */

declare(strict_types=1);

```



```

        <th>Name</th>
        <th>Email</th>
        <th>Phone</th>
        <th>Course</th>
        <th>Year</th>
        <th>Message</th>
    </tr>
</thead>
<tbody>
    <?php foreach ($records as $record): ?>
        <tr>
            <td><?= e((string) ($record['id'] ?? ''))
?></td>
            <td><?= e((string) ($record['submitted_at'] ??
'')) ?></td>
            <td><?= e((string) ($record['full_name'] ??
'')) ?></td>
            <td><?= e((string) ($record['email'] ?? ''))
?></td>
            <td><?= e((string) ($record['phone'] ?? ''))
?></td>
            <td><?= e((string) ($record['course'] ?? ''))
?></td>
            <td><?= e((string) ($record['year'] ?? ''))
?></td>
            <td><?= e((string) ($record['message'] ?? ''))
?></td>
        </tr>
    <?php endforeach; ?>
</tbody>
</table>
</div>
<?php endif; ?>
</main>
</body>
</html>

```

### 3.config.php:

```

<?php
/**
 * Practical 7 - Configuration
 * Central place for storage paths, field rules and app settings.
 */

declare(strict_types=1);

const APP_NAME          = 'Student Hub - Registration';
const DATA_DIR          = __DIR__ . '/data';
const CSV_FILE           = DATA_DIR . '/registrations.csv';
const JSON_FILE          = DATA_DIR . '/registrations.json';
const CSV_HEADER         = ['id', 'submitted_at', 'full_name', 'email',
                             'phone', 'course', 'year', 'message', 'ip_address'];

/** Field rules used for both validation and sanitisation. */
const FIELD_RULES = [
    'full_name' => ['required' => true, 'min' => 3, 'max' => 60, 'label'
=> 'Full name'],
    'email'      => ['required' => true, 'max' => 120, 'label'
=> 'Email'],
    'phone'      => ['required' => true, 'min' => 7, 'max' => 15, 'label'
=> 'Phone number'],
    'course'     => ['required' => true, 'max' => 60, 'label'
=> 'Course'],
    'year'       => ['required' => true, 'label' =>
'Year of study'],
    'message'    => ['required' => false, 'min' => 10, 'max' => 500,
'label' => 'Message'],
];

/** Allowed values for select inputs (checked again on the server). */
const ALLOWED_COURSES = ['BCA', 'BBA', 'BCS', 'BCom', 'MCA', 'MSc'];
const ALLOWED_YEARS   = ['1', '2', '3', '4', '5'];

ensure_data_dir();

/** Create the storage directory (and .htaccess guard) if it does not
exist. */
function ensure_data_dir(): void

```

```

{
    if (!is_dir(DATA_DIR)) {
        mkdir(DATA_DIR, 0775, true);
    }

    $htaccess = DATA_DIR . '/.htaccess';
    if (!file_exists($htaccess)) {
        file_put_contents(
            $htaccess,
            "# Practical 7 - deny direct web access to stored records\n"
            . "<IfModule mod_authz_core.c>\n    Require all\n"
            . "denied\n</IfModule>\n"
            . "<IfModule !mod_authz_core.c>\n    Order deny,allow\n"
            . "Deny from all\n</IfModule>\n"
            );
    }
}

```

#### 4.runtest.php

```

<?php
/**
 * Practical 7 - Automated test cases for validation, sanitisation and
 * storage.
 * Run from the project root:  php tests/run_tests.php
 */

declare(strict_types=1);

require_once __DIR__ . '/../includes/bootstrap.php';

$passed = 0;
$failed = 0;

function check(string $label, bool $condition): void
{
    global $passed, $failed;

    if ($condition) {

```

```

        $passed++;
        echo "[PASS] $label\n";
    } else {
        $failed++;
        echo "[FAIL] $label\n";
    }
}

function valid(array $overrides = []): array
{
    return array_merge([
        'full_name' => 'Aisha Khan',
        'email'      => 'aisha.khan@example.com',
        'phone'      => '9876543210',
        'course'     => 'BCA',
        'year'       => '2',
        'message'    => 'I am interested in joining the student hub.',
    ], $overrides);
}

echo "=== Practical 7 - Sanitisation tests ===\n";
check('Trims surrounding whitespace', Validator::sanitizeText("  Aisha  ")
=== 'Aisha');
check('Strips HTML tags',
Validator::sanitizeText('<script>alert(1)</script>Bob') ===
'alert(1)Bob');
check('Removes null bytes', Validator::sanitizeText("Ais\0ha") ===
'Aisha');
check('Collapses internal whitespace', Validator::sanitizeText("Aisha
Khan") === 'Aisha Khan');
check('Email lowercased', Validator::sanitizeEmail('  Aisha@Example.COM ')
=== 'aisha@example.com');
check('Multiline keeps newlines',
str_contains(Validator::sanitizeMultiline("line1\n\n\n\nline2"),
"\n\nline2"));

echo "\n=== Practical 7 - Validation tests ===\n";

$r = Validator::validate(valid());
check('Valid submission has no errors', $r['errors'] === []);

```

```
$r = Validator::validate(valid(['full_name' => '']));  
check('Empty required name is rejected',  
isset($r['errors']['full_name']));  
  
$r = Validator::validate(valid(['full_name' => 'A']));  
check('Name shorter than 3 chars is rejected',  
isset($r['errors']['full_name']));  
  
$r = Validator::validate(valid(['full_name' => 'Bad<script>Name']));  
check('Tags in name are sanitised away, not stored',  
$r['data']['full_name'] === 'BadName');  
check('Sanitised name is then accepted',  
!isset($r['errors']['full_name']));  
  
$r = Validator::validate(valid(['full_name' => 'Bad <b>Name</b>!']));  
check('Name with illegal characters is rejected',  
isset($r['errors']['full_name']));  
  
$r = Validator::validate(valid(['email' => 'not-an-email']));  
check('Invalid email is rejected', isset($r['errors']['email']));  
  
$r = Validator::validate(valid(['phone' => 'abc']));  
check('Invalid phone is rejected', isset($r['errors']['phone']));  
  
$r = Validator::validate(valid(['course' => 'HACKED']));  
check('Course not in whitelist is rejected',  
isset($r['errors']['course']));  
  
$r = Validator::validate(valid(['year' => '9']));  
check('Year not in whitelist is rejected', isset($r['errors']['year']));  
  
$r = Validator::validate(valid(['message' => 'short']));  
check('Message under 10 chars is rejected',  
isset($r['errors']['message']));  
  
$r = Validator::validate(valid(['message' => str_repeat('a', 501)]));  
check('Message over 500 chars is rejected',  
isset($r['errors']['message']));
```

```

echo "\n=== Practical 7 - Storage tests ===\n";

$csvPath = CSV_FILE;
$jsonPath = JSON_FILE;
$csvBackup = is_file($csvPath) ? file_get_contents($csvPath) : null;
$jsonBackup = is_file($jsonPath) ? file_get_contents($jsonPath) : null;

@unlink($csvPath);
@unlink($jsonPath);

Storage::append(['submitted_at' => date('Y-m-d H:i:s'), 'full_name' =>
'Test One', 'email' => 'one@example.com',
                'phone' => '9000000001', 'course' => 'BCA', 'year' =>
'1', 'message' => 'First test record', 'ip_address' => '127.0.0.1']);
check('CSV file created on first write', is_file($csvPath));
check('JSON file created on first write', is_file($jsonPath));
check('First record has id 1', Storage::readJson()[0]['id'] === 1);

Storage::append(['submitted_at' => date('Y-m-d H:i:s'), 'full_name' =>
'Test, Two "quoted"', 'email' => 'two@example.com',
                'phone' => '9000000002', 'course' => 'BBA', 'year' =>
'3', 'message' => 'Second, with comma', 'ip_address' => '127.0.0.1']);
check('Second record has id 2', Storage::readJson()[1]['id'] === 2);
check('JSON count is 2', Storage::count() === 2);
check('CSV header written once', count(Storage::readCsv()) === 2);
check('CSV handles commas and quotes', Storage::readCsv()[1]['full_name']
=== 'Test, Two "quoted"');
check('JSON is valid JSON', is_array(json_decode((string)
file_get_contents($jsonPath), true)));

$before = count(Storage::readJson());
Validator::validate(valid(['email' => 'one@example.com']));
$r = Validator::validate(valid(['email' => 'one@example.com']));
check('Duplicate email is rejected', isset($r['errors']['email']));
check('Storage count unchanged after duplicate attempt',
count(Storage::readJson()) === $before);

if ($csvBackup !== null) { file_put_contents($csvPath, $csvBackup); } else
{ @unlink($csvPath); }

```

```

if ($jsonBackup !== null) { file_put_contents($jsonPath, $jsonBackup); }
else { @unlink($jsonPath); }

echo "\n=== Practical 7 - CSRF tests ===\n";
$_SESSION = [];
$token = Csrf::token();
check('Token is 64 hex chars', preg_match('/^[a-f0-9]{64}$/ ', $token) ===
1);
check('Same token returned within a session', Csrf::token() === $token);
check('Wrong token is rejected', Csrf::validate('bad-token') === false);
check('Correct token is accepted', Csrf::validate($token) === true);
check('Token cannot be replayed', Csrf::validate($token) === false);

echo "\n-----\n";
echo "Passed: $passed | Failed: $failed\n";
exit($failed === 0 ? 0 : 1);

```

## 5.generateSample.php:

```

<?php
/**
 * Practical 7 - Generate a few sample records so the CSV/JSON files
 * can be submitted as part of the lab report.
 * Run: php tests/generate_sample_data.php
 */

declare(strict_types=1);

require_once __DIR__ . '/../includes/bootstrap.php';

$total = Storage::count();
if ($total > 0) {
    echo "Sample data already present ($total record(s)) - nothing to
do.\n";
    exit(0);
}

$sample = [

```

```

    ['Aisha Khan',      'aisha.khan@example.com',    '9876543210', 'BCA',
'2', 'Looking forward to the coding club activities.'],
    ['Rohit Sharma',    'rohit.sharma@example.com', '9812345678', 'BCS',
'3', 'Please add me to the newsletter.'],
    ['Priya Menon',     'priya.menon@example.com',   '9900112233', 'MCA',
'1', 'Interested in the internship cell.'],
    ['Daniel Fernandes', 'daniel.f@example.com',     '9700334455', 'BBA',
'4', 'Requesting information about hostel facilities.'],
    ['Sneha Iyer',      'sneha.iyer@example.com',    '9655443322',
'BCom', '2', 'How can I join the photography society?'],
];

foreach ($sample as [$name, $email, $phone, $course, $year, $message]) {
    Storage::append([
        'submitted_at' => date('Y-m-d H:i:s'),
        'full_name'     => $name,
        'email'         => $email,
        'phone'         => $phone,
        'course'        => $course,
        'year'          => $year,
        'message'       => $message,
        'ip_address'    => '::$1',
    ]);
}

echo "Created " . Storage::count() . " sample records in
data/registrations.csv and data/registrations.json\n";

```

## Style.css

```

:root {
    --bg: #0f172a;
    --card: #ffffff;
    --ink: #1f2937;
    --muted: #6b7280;
    --line: #d8dde5;
    --brand: #2563eb;
    --ok-bg: #ecfdf5; --ok-line: #34d399; --ok-ink: #065f46;
    --err-bg: #fef2f2; --err-line: #f87171; --err-ink: #991b1b;
}

```

```
--info-bg: #eff6ff; --info-line: #60a5fa; --info-ink: #1e40af;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  padding: 32px 16px;
  font: 16px/1.55 "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  color: var(--ink);
  background: var(--bg);
}

.card {
  max-width: 720px;
  margin: 0 auto;
  background: var(--card);
  border-radius: 14px;
  padding: 28px 32px 32px;
  box-shadow: 0 18px 40px rgba(0, 0, 0, .35);
}

.card.wide { max-width: 1040px; }

h1 { margin: 0 0 6px; font-size: 26px; }
.lead { margin: 0 0 20px; color: var(--muted); font-size: 15px; }
.meta { color: var(--muted); font-size: 14px; }

.alert { padding: 12px 16px; border-radius: 10px; border-left: 5px solid;
margin-bottom: 18px; }
.alert-success { background: var(--ok-bg); border-color: var(--ok-line);
color: var(--ok-ink); }
.alert-error { background: var(--err-bg); border-color: var(--err-line);
color: var(--err-ink); }
.alert-info { background: var(--info-bg); border-color:
var(--info-line); color: var(--info-ink); }

.error-summary { margin: 0 0 20px; padding-left: 20px; color:
var(--err-ink); }
.error-summary a { color: inherit; }
```

```
.field { margin-bottom: 16px; }
.row { display: flex; gap: 16px; flex-wrap: wrap; }
.row .field { flex: 1 1 200px; }

label { display: block; font-weight: 600; margin-bottom: 6px; font-size:
14px; }
.req { color: #dc2626; }

input[type="text"], input[type="email"], select, textarea {
  width: 100%;
  padding: 10px 12px;
  border: 1px solid var(--line);
  border-radius: 8px;
  font: inherit;
  background: #fff;
  color: var(--ink);
}
input:focus, select:focus, textarea:focus {
  outline: none;
  border-color: var(--brand);
  box-shadow: 0 0 0 3px rgba(37, 99, 235, .18);
}
textarea { resize: vertical; }

.field.has-error input, .field.has-error select, .field.has-error textarea
{ border-color: var(--err-line); }
.error { margin: 6px 0 0; color: var(--err-ink); font-size: 13px;
font-weight: 600; }

.actions { display: flex; gap: 12px; flex-wrap: wrap; margin-top: 20px; }
.btn {
  display: inline-block;
  padding: 10px 18px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #fff;
  color: var(--ink);
  font: inherit;
  font-weight: 600;
  text-decoration: none;
}
```

```

        cursor: pointer;
    }
    .btn:hover { border-color: var(--brand); color: var(--brand); }
    .btn-primary { background: var(--brand); border-color: var(--brand);
color: #fff; }
    .btn-primary:hover { background: #1d4ed8; color: #fff; }

    .summary { display: grid; grid-template-columns: max-content 1fr; gap: 6px
18px; margin: 0 0 8px; }
    .summary dt { font-weight: 600; color: var(--muted); }
    .summary dd { margin: 0; }

    .table-wrap { overflow-x: auto; border: 1px solid var(--line);
border-radius: 10px; }
    table { border-collapse: collapse; width: 100%; font-size: 14px; }
    th, td { padding: 10px 12px; text-align: left; border-bottom: 1px solid
var(--line); vertical-align: top; }
    th { background: #f3f4f6; font-weight: 600; white-space: nowrap; }
    tbody tr:last-child td { border-bottom: none; }
    tbody tr:nth-child(even) { background: #fafafa; }

    code { background: #f3f4f6; padding: 1px 5px; border-radius: 4px;
font-size: 13px; }

```

**Output:**

html

# Registration Form

Practical 7 - PHP Form Processing

Name \*